

Learning to Use Sparse Native-KV Memory: A Controlled Adaptation Ladder for Progressive Retrieval Attention

Jorge Simao

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 25, 2026

Abstract

Progressive Retrieval Attention (PRA) can route to external references and materialize their layer-native key/value (K/V) state, yet a transformer trained only with ordinary self-attention need not know how to use that state. We isolate this consumer-learning problem in a controlled associative-chain model. Reference identities are fixed by an oracle, reference K/V remains inside the autograd graph, and memory joins local K/V under the model’s ordinary attention softmax. Starting from one matched LocalSA checkpoint per seed, we compare Frozen PRA, three LoRA scopes, full-weight continued training, and PRA-native training from scratch. Each regime is evaluated with no memory, matched distractors, exact evidence, and the whole parent. Across 5 paired seeds, the best change in the correct-memory margin effect over Frozen is 9.512 for Native scratch. Evidence-only accuracy rises from 0.303 Frozen to 0.821 with Consumer LoRA and 0.992 from scratch, while whole-parent accuracy plateaus near one half. The controlled gate passes 3 preregistered criteria and yields the decision `pass_to_tier1`. These results concern oracle-selected memory consumption in a sub-million-parameter diagnostic model. They do not establish learned routing, language-model scaling, or deployment efficiency. We additionally specify and exercise the complete read-side external-memory lifecycle—cold descriptors, authenticated lazy admission, scoped warm native encodings, and hot K/V—and prepare exact-slot Gemma 3 1B adaptation. The systems fixture validates lifecycle and security behavior, not answer quality; the Gemma training run is held behind a compute gate.

1 Introduction

Retrieval-augmented systems are often evaluated as if selecting the right text were sufficient. Selection is only one boundary. The downstream transformer must also turn retrieved state into a useful residual update without being distracted by irrelevant state. Architectures trained with retrieval in the loop, including REALM, RAG, and RETRO, make this interface part of learning [4, 6, 2]. A frozen transformer retrofitted with a new memory path has no such guarantee.

PRA makes this distinction unusually visible. It stores historical token K/V in the same representation used by a transformer’s attention layers, while compact gists are only an address index. Once a reference is selected, full token K/V is placed before local K/V under a joint softmax. This avoids a separate cross-attention value space, but native format does not imply native use. Queries, memory keys, values, output projections, and downstream feed-forward layers were all optimized for a different context topology.

This paper asks a deliberately narrower question before spending compute on a larger language model:

When memory identity and physical K/V are correct, how much transformer plasticity is needed before sparse memory improves the answer?

The experiment is a causal gate, not a benchmark race. We reuse randomized associative chains, hold the PRA topology fixed, and change only adaptation capacity. The design follows a strict ladder from no trainable parameters through targeted low-rank updates [5] to full and from-scratch training. Four memory interventions separate useful state from the generic effect of adding tokens. Layerwise attention and representation-portability measurements then show where any output change enters the network.

PRA also needs a boundary before the native cache. A document on the Web, in an object store, or in a private registry is not yet transformer memory. It first exists as cheap metadata; after admission it may be fetched and encoded; only a selected subset becomes accelerator-resident K/V. Treating those states as one opaque “external memory” hides authorization, invalidation, and first-use cost. We therefore add the read-side hierarchy required by the subsequent pretrained experiment while keeping memory writes and consolidation out of scope.

Contributions. We provide (i) a differentiable native-KV training path that preserves the same joint attention operation used at inference; (ii) a matched six-rung PRA adaptation ladder plus GlobalSA and LocalSA controls; (iii) causal metrics linking memory identity to attention, residual state, answer margin, and prediction; and (iv) a stage-gated result that explicitly decides whether larger-model work is warranted; and (v) a resolver-neutral cold/warm/hot lifecycle with executable security invariants and an exact native-global-slot Gemma training gate.

2 Why a Native Interface May Still Be Foreign

Consider a decoder layer ℓ with local hidden state $H_L^{(\ell)}$ and a separately encoded memory sequence $H_M^{(\ell)}$. PRA reuses the layer’s ordinary projections:

$$Q_L^{(\ell)} = W_Q^{(\ell)} \text{LN}(H_L^{(\ell)}), \quad (1)$$

$$K_M^{(\ell)}, V_M^{(\ell)} = W_{K,V}^{(\ell)} \text{LN}(H_M^{(\ell)}), \quad (2)$$

$$K_L^{(\ell)}, V_L^{(\ell)} = W_{K,V}^{(\ell)} \text{LN}(H_L^{(\ell)}). \quad (3)$$

After the configured positional transform, memory and local keys share one normalizer:

$$A^{(\ell)} = \text{softmax} \left(\frac{Q_L^{(\ell)} [K_M^{(\ell)}; K_L^{(\ell)}]^\top}{\sqrt{d_h}} + \mathcal{M}_{\text{causal}} \right), \quad U^{(\ell)} = W_O^{(\ell)} A^{(\ell)} [V_M^{(\ell)}; V_L^{(\ell)}]. \quad (4)$$

Memory positions are visible to every local query; the local block remains causal.

This operation introduces no cross-attention projection. Nevertheless, four mismatches can remain. A query may not score remote keys correctly. Memory keys may depend too strongly on the context in which they were encoded. Memory values may point in directions the downstream residual stream does not interpret. Later layers may erase a useful update. Frozen evaluation conflates all four. The adaptation ladder opens them in a controlled order.

3 Differentiable Oracle-Memory Training

Offline PRA caches detach reference state so one encoding can serve many queries. That is the correct inference boundary but the wrong training boundary. During Gate 0, the selected reference tokens are encoded in the same forward graph as the query. Immediately before each PRA consumer layer, the model captures $K_M^{(\ell)}, V_M^{(\ell)}$ with detachment disabled. The query then consumes those tensors through the native joint softmax above. Gradients can reach Q/K/V/O, reference production, and subsequent consumers according to the selected regime.

```
memory_h = embed(reference_tokens)
for layer in decoder:
    if layer is PRA:
        memory_kv[layer] = project_kv(LN(memory_h), detach=False)
        memory_h = layer.local_forward(memory_h)

query_h = embed(query_tokens)
for layer in decoder:
    if layer is PRA:
        update = native_joint_attention(LN(query_h), memory_kv[layer])
        query_h = query_h + update
        query_h = query_h + FFN(LN(query_h))
    else:
        query_h = layer.local_forward(query_h)
```

Oracle selection is intentional. Training a router, materializer, and consumer at once would make a negative output result uninterpretable. Evidence masks are used only after the forward pass to divide final-token attention into evidence and distractor mass. They never enter model input or loss.

4 Hierarchical External Memory Admission

The differentiable experiment begins with already selected token memory. A deployed read path begins earlier. For an external document D , let $g_{\text{ext}}(D)$ denote a cheap descriptor such as a title, sparse index, embedding, summary, or content hash. It is distinct from both a model-native PRA gist and the detailed K/V later supplied to attention:

$$g_{\text{ext}}(D) \longrightarrow \text{admit/fetch} \longrightarrow g_{\text{PRA}}(D), K_D, V_D \longrightarrow K_{D,S}, V_{D,S}. \quad (5)$$

The final subset S is the materialization decision. External retrieval may be BM25, dense, hybrid, a manual URI, or a service call; it is modular and need not be differentiable in Paper 4.

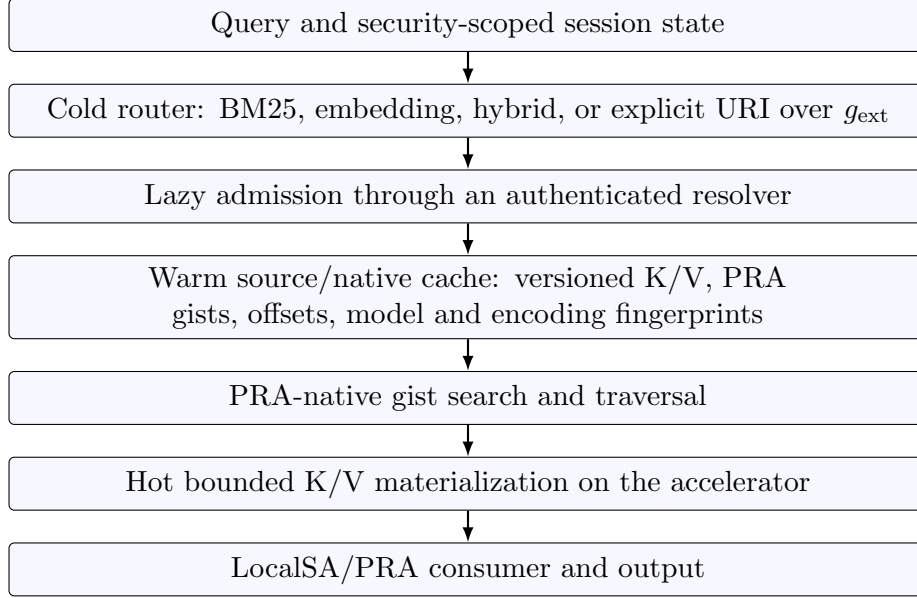


Figure 1: Paper 4’s complete read-side path. Standard RAG stops after fetching text and concatenates it into a prompt. RAG–PRA uses the same cold retriever, then creates reusable native memory and selectively exposes K/V. Paper 4.5 owns the separate write/consolidation path.

We implement four encoding modes. **eager** fetches and encodes at registration; **lazy** waits for admission; **cached_lazy** reuses a valid warm encoding; and **metadata_only** may route but cannot become native memory without a later permission/fetch transition. Resolvers implement asynchronous **stat**, **fetch**, and **external_gist** operations behind scheme plugins. HTTP, cloud stores, local files, Hugging Face, and private registries therefore share one lifecycle rather than entering PRA as special cases.

Cache reuse is keyed by security scope, canonical URI, source version, model and tokenizer fingerprints, and encoding/position configuration. Every reuse calls the resolver again to revalidate authorization. Credentials remain opaque provider handles passed only from the session’s authorization context to the resolver; they never enter prompts, model inputs, cache keys, logs, or artifacts. Session teardown removes ephemeral entries, and global reuse is legal only for resources explicitly marked public.

4.1 Read-side mechanism check

A deterministic eight-resource fixture exercises the entire transition and records each latency component separately. Its simple lexical descriptor router admits the required document in 0.875 of cases, with false-admission rate 0.125. One ambiguous descriptor accounts for the miss; this is useful because it demonstrates that document admission recall and conditional within-document evidence recovery must be reported separately.

State	Work paid on access	Mean (ms)	Quality meaning
Cold	route, fetch, encode, materialize	38.98	document availability only
Warm	route, cached native encode, materialize	13.58	document availability only
Hot	route, cached accelerator K/V	0.59	document availability only

Table 1: Synthetic lifecycle decomposition over 8 resources. Delays include a deterministic resolver fixture and operating-system scheduling; they validate cost ownership and cache ordering, not production latency or language-model answer quality.

Executable tests verify stale-version invalidation, private cross-user cache isolation, public-cache restrictions, ACL revalidation, credential redaction, metadata-only behavior, and ephemeral teardown. The same external router is used for the standard-RAG and lazy-RAG-PRA rows in the generated artifacts, preventing first-stage retrieval differences from being attributed to the native memory interface. The latency artifact also separates fetched bytes, newly encoded tokens, warm native-memory bytes, hot active K/V bytes, and a materialization transfer-time proxy. Actual answer EM/F1 and neural encoding FLOPs remain pending the pretrained training gate.

5 Controlled Experiment

5.1 Task and matched architectures

Each example draws fresh entity and relation symbols, builds a path of one to four facts, adds relation-sharing and label-relation distractors, and asks for the final entity. Stable entity-to-answer memorization is therefore unavailable. Facts are independently addressable references. The complete parent trains matched GlobalSA and LocalSA models; PRA receives a short query as direct context and selected facts as external K/V.

All models have 64 hidden dimensions, eight decoder layers, four heads, RoPE, and a 256-token operation limit. LocalSA uses a 16-token window. PRA replaces attention at zero-indexed layers 3 and 7, leaving LocalSA/FFN assimilation between memory interventions. This model is far below the nominal < 100M Tier-0 ceiling: the goal is mechanistic discrimination at low cost.

5.2 Adaptation ladder

GlobalSA and LocalSA use identical random initial weights within each seed; only their attention windows differ. Every converted PRA regime starts from that seed’s trained LocalSA checkpoint.

1. **Frozen:** conversion only; no trainable parameters.
2. **Consumer LoRA:** low-rank updates to PRA Q/O and the immediately following FFNs.
3. **Interface LoRA:** Consumer LoRA plus PRA K/V.
4. **Broad LoRA:** low-rank updates to every decoder linear module.
5. **Full weight:** all converted-model parameters continue training.
6. **Native scratch:** the same PRA topology is present from initialization and all weights train end to end.

LoRA rank is eight with scale $\alpha/r = 2$. Baselines train for 800 steps; converted trainable regimes continue for 500; native scratch trains for the matched total of 1,300. We use five fixed model seeds, 4,096 deterministic training examples, and 512 held-out examples. The objective is answer-token cross entropy. No held-out result selects a checkpoint.

5.3 Causal interventions and metrics

For answer token y and final logits z , define the margin

$$m = z_y - \max_{v \neq y} z_v. \quad (6)$$

The primary usefulness effect compares correct and equally sized incorrect memory:

$$G_m = m_{\text{evidence}} - m_{\text{matched distractor}}. \quad (7)$$

We also report no-memory, evidence-only, and whole-parent conditions. Memory modularity is measured by

$$\Delta\text{NLL} = \text{NLL}_{\text{evidence}} - \text{NLL}_{\text{parent}}, \quad (8)$$

with analogous accuracy and margin differences. Evidence selectivity is evidence attention mass divided by total memory attention mass in the whole-parent condition.

To test whether memory objects become less dependent on neighboring facts, we encode the same evidence tokens alone and in the whole parent. At PRA-facing layer ℓ , key portability is

$$D_K^{(\ell)} = 1 - \cos\left(K_{e,\text{alone}}^{(\ell)}, K_{e,\text{parent}}^{(\ell)}\right), \quad (9)$$

averaged over heads, tokens, examples, and seeds; values are measured likewise.

6 Results

6.1 Adaptation capacity and causal usefulness

Table 2 reports the complete ladder. “Margin gain” is G_m ; its uncertainty column is the across-seed standard deviation. The best paired improvement over Frozen is 9.512 for Native scratch. This comparison is more diagnostic than evidence-versus-no-memory because it controls the number of imported tokens. Accuracy supplies the same practical picture. Frozen reaches 0.303 on evidence-only memory. Consumer, Interface, Broad, Full, and Native reach 0.821/0.875/ 0.948/0.964/ 0.992, respectively. Thus even the narrowest trainable consumer scope recovers a large part of the oracle-memory ceiling, while additional capacity continues to increase margin and confidence.

6.2 Memory modularity and layer specialization

Whole-parent accuracy also improves: Frozen reaches 0.291, while Consumer, Interface, Broad, Full, and Native reach 0.481/0.495/ 0.480/0.484/ 0.477. This is a real noisy-memory gain, but it is much smaller than the evidence-only gain. No regime reduces the absolute evidence-versus-parent NLL gap; the least unfavorable change is -0.459 under Consumer LoRA. Gate 0 therefore rejects the strong modularity hypothesis. Whole-parent attention provides a stricter view: an adapted consumer should allocate its fixed memory mass to evidence rather than merely increase attention to every imported token. The strongest paired selectivity change is 0.011 for Native scratch. It has the same direction in all seeds but is small in magnitude. Adaptation mainly changes what the network can do with clean memory; it does not yet create a strong token-level filter among distractors.

Regime	Trainable	Margin gain	Δ NLL	Selectivity
Frozen	0.000	1.480 ± 0.271	-0.197	0.332
Consumer LoRA	0.031	3.307 ± 0.191	-0.656	0.329
Interface LoRA	0.040	3.924 ± 0.456	-0.754	0.330
Broad LoRA	0.142	4.957 ± 0.318	-0.891	0.333
Full weight	1.000	6.470 ± 0.259	-1.062	0.333
Native scratch	1.000	10.992 ± 0.493	-1.583	0.343

Table 2: Five-seed controlled adaptation ladder. Trainable is the parameter fraction exposed during adaptation; Δ NLL compares evidence-only with the whole parent; selectivity is evidence’s share of whole-parent memory attention.

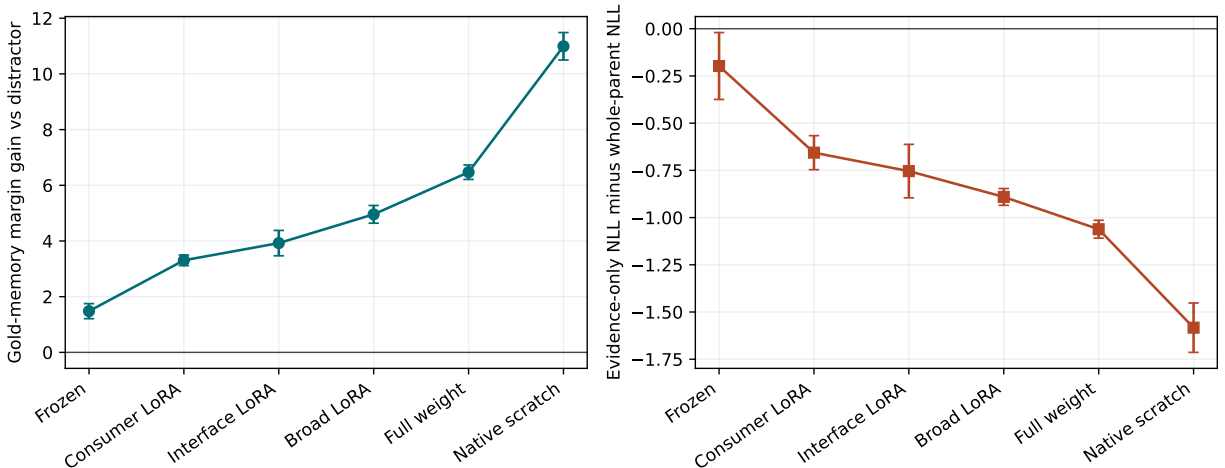


Figure 2: Left: answer-margin benefit of exact evidence over matched distractor memory. Right: evidence-only versus whole-parent NLL. Error bars are across-seed standard deviations, not confidence intervals.

6.3 Retention and chain depth

Converted LoRA models largely retain the original no-memory full-context path. Accuracy is 0.400 Frozen, 0.391 Consumer, 0.391 Interface, and 0.383 Broad. Full-weight continuation reaches 0.374. Native scratch reaches only 0.184 because it was trained through the sparse-memory path rather than the original full-context interface. The strongest oracle ceiling is therefore not the safest direct replacement for a pretrained model.

Evidence-only accuracy remains high through the four-hop stratum: Full and Native both reach 0.995 and 0.995, respectively. We do not interpret the upward shape of some curves as a causal depth benefit. Each stratum is a different randomized sample, and the generator was designed to prevent label leakage rather than calibrate equal difficulty by depth.

6.4 Representation portability

Figure 5 measures a different property from output quality. A low value means the same evidence token produces similar keys when encoded alone or amid distractors. A regime can improve answer use without making keys portable, or make keys portable without teaching downstream layers to interpret values. Reporting both prevents an output-only explanation from being mistaken for a

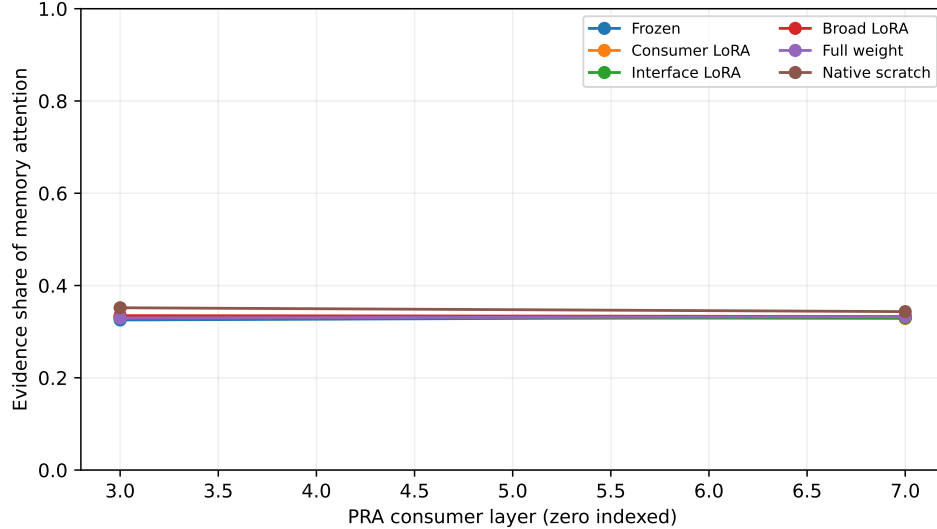


Figure 3: Evidence share of final-token memory attention at each PRA consumer. Evidence labels are diagnostic only and were not available to the model.

representation result.

6.5 Gate decision

The native-scratch evidence-memory accuracy minus LocalSA full-context accuracy is 0.592 on average. In total, 3 preregistered signals pass, producing `pass_to_tier1`. With five paired seeds, these results remain descriptive: even a same-direction effect in all seeds has a minimum exact two-sided sign-flip $p = 0.0625$. We therefore use the gate to decide the next experiment, not to claim a population-level scaling law.

7 Pretrained Gate: Exact-Slot Gemma-PRA

The next model is the pinned `google/gemma-3-1b-it` checkpoint at revision `dcc83ea8...cf78752`; the complete immutable hash is recorded in the machine-readable gate artifact. Its configuration declares 26 decoder layers, a 512-token local window, and full-attention layers at zero-based indices 5, 11, 17, and 23. The primary conversion replaces exactly those four global slots with PRA and leaves all 22 local-attention layers unchanged. The map is generated from the checkpoint’s `layer_types`; it is not a hard-coded Gemma-family assumption.

Rank-8 parameter estimates are approximately 0.91M for Consumer LoRA, 1.00M for Interface LoRA, and 6.52M for Broad LoRA; rank 32 is retained as the single compact capacity control. These are architecture-derived estimates and will be replaced by loaded-model counts in the measured run. Full-weight mixed-precision Adam has an 11.2 GiB lower-bound state cost for this one-billion-parameter checkpoint before activations, temporary buffers, reference K/V, and checkpoint staging. The available GTX 950M has 4 GiB. In accordance with the benchmark-before-budget rule, G2–G5 training is therefore not launched here: the next action is to configure distributed or larger-memory execution, then run 100–500 measured steps before approving any longer schedule.

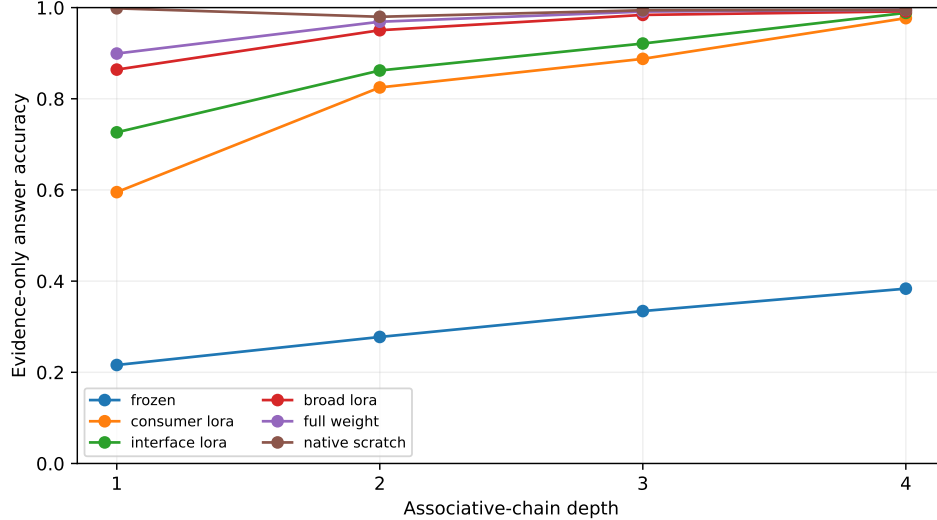


Figure 4: Evidence-only accuracy by associative-chain depth. Adaptation gains persist through depth four; cross-depth slopes are descriptive because difficulty is not calibrated across strata.

Stage	Same-checkpoint intervention	Status
G0	Native Gemma local/global schedule	measured baseline pending
G1	Frozen exact-slot Gemma-PRA	measured baseline pending
G2	Consumer LoRA: PRA Q/O and following MLP	implementation and scope tests ready
G3	Interface LoRA: PRA Q/K/V/O and following MLP	implementation and scope tests ready
G4	Broad decoder LoRA	implementation and scope tests ready
G5	Full-weight exact-slot Gemma-PRA	compute benchmark pending
G6	Gemma-like PRA-native scratch	gated on G5

Table 3: The pretrained adaptation ladder. No row is reported as an empirical quality result until its fixed-data training and evaluation artifacts exist.

8 Related Work

Sparse-attention methods reduce which token pairs interact inside a sequence [1, 10, 8]. Recurrent and compressed-memory transformers carry state across segments [3, 7]; Memorizing Transformers retrieve stored representations [9]. PRA differs in separating compact routing metadata from full-detail, layer-native K/V materialized under a bounded operation. This paper studies the learning interface after selection rather than proposing another retrieval score.

Retrieval-augmented language models train retrieval and generation jointly or in coordinated stages [4, 6, 2]. Our oracle stage is intentionally simpler. It provides a causal upper-bound question: if the correct physical memory is already present, can adaptation make it useful? A negative answer would make router optimization premature; a positive answer justifies reconnecting learned routing at the next gate.

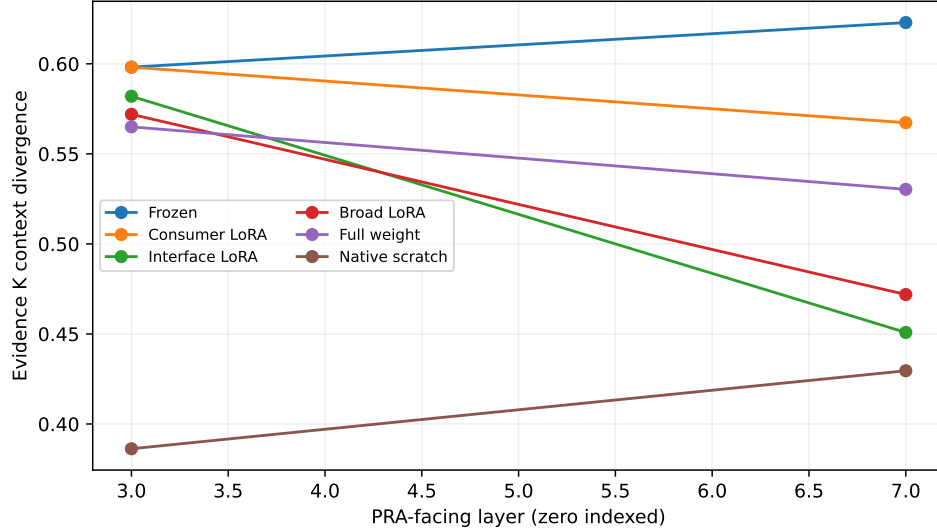


Figure 5: Context dependence of evidence keys at PRA-facing layers. Values near zero indicate more portable representations across evidence-only and parent encodings.

9 Limitations and Next Gate

The controlled vocabulary, answer-only objective, sub-million-parameter model, and short schedule do not demonstrate ordinary language modeling. Oracle memory removes identity errors and does not test top- k recall, graph traversal, or materialization budgets. The reported attention masses are correlates of use; the matched-distractor margin is the stronger causal output control. We do not measure production throughput because references remain in the training graph.

The next tier should preserve the same intervention structure while adding an ordinary-language objective, local-only cases, remote-memory cases, multi-resource evidence, and distractor-rich memory. Consumer and Interface LoRA are the conservative first measured Gemma conditions because they capture most of the toy evidence-only gain while limiting trainable state. Learned routing is reintroduced alongside fixed-oracle, shuffled, disabled, matched-distractor, and whole-parent controls. The lifecycle fixture uses synthetic descriptors and simulated resolver latency; it does not establish public-Web retrieval quality, production cache speed, or authenticated cloud-provider correctness. Gemma quality retention, memory modularity, active-global K/V reduction, and logical memory scaling all remain unmeasured until the compute gate is funded.

10 Conclusion

Sparse memory has two learning problems: finding the right state and making that state computationally useful. By fixing the first, this study measures the second directly. The differentiable native-KV path and matched adaptation ladder turn a previously ambiguous frozen result into a testable training question. The outcome supports a stage-gated research program in which consumer plasticity is established before router complexity and model scale are increased. The accompanying read-side hierarchy also makes the deployment premise explicit: cold external resources become scoped, versioned native memory lazily, while only a bounded hot subset reaches attention. The completed toy gate justifies the pretrained experiment; it does not substitute for it.

References

- [1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.
- [3] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.
- [4] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Retrieval augmented language model pre-training. In *International Conference on Machine Learning*, 2020.
- [5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [7] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.
- [8] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021. Preprint: <https://arxiv.org/abs/2003.05997>.
- [9] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.
- [10] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.